

【作品名】 迷子防止センサー

【制作期間】 2026 年 4 月下旬～5 月中旬

【開発環境・使用部品】

開 発 環 境 (I D E) : Arduino IDE

使 用 言 語 : C/C++(Arduino 言語)

マ イ コ ン ボ ー ド : ESP32-WROVER(2 台)

ハードウェア・電子部品:

RGB LED

パッシブブザー

I2C 接続小型キャラクタ LCD モジュール

ブレッドボード、ジャンパ線、各種抵抗器

【機能】

- 共通機能
- ・ ESP-NOW を用いて親機・子機間でのデータ送受信を行う Peer to Peer 通信
 - ・ 常時 RSSI を監視し、警告を-50dBm・-70dBm、通信途絶の 3 段階に分け警告音を制御
 - ・ LCD を設置し、通信強度を視覚的に表示。親機・子機それぞれで異なる表示方法を採用

- 子 機
- ・ RSSI のしきい値に応じて RGB LED の発行色を変化させ、通信状態および警告段階を視覚的に確認できるよう設計

【学習のきっかけ・開発動機】

2026 年 4 月中旬から下旬にかけ、Arduino IDE とマイコン (ESP32) を使用した組み込み開発の基礎を学習しました。主に LED やブザー、可変抵抗器の制御を学ぶ中で、ESP32 に内蔵されている Bluetooth や Wi-Fi 機能の接続設定コードに触れる機会があり、この通信機能を活用すれば、さらに実用的なモノづくりができるのではないかと興味を持ったことが始まりです。

以上の学習をもとに Wi-Fi 接続を使用したセンサーを活用し、実用的な IoT システムの開発に挑戦したいと考え「子供の迷子防止センサー」の制作を決めました。私自身子供がおり、ほんの少しでも手や目を離すとすぐになくなってしまふ、そういう経験をもとに親の手元から一定距離離れた際アラートを発するシステムがあれば子供の迷子を未然に防げるのではないかと考えたからです。これを実現するため、1 台を子機、もう 1 台を親機に見立てた 2 台の ESP32 による無線通信システムの開発をスタートしました。

【学習作品制作ロードマップ】

作品を実現させるための学習フローを以下の表にまとめました

項 目	内 容（授業内容【 】 先行学習【 】）	実施状況
GPIO 基礎	・ INPUT/OUTPUT モードの理解	学習済
	・ HIGH/LOW によるデジタル出力の基礎	学習済
	・ サンプルプログラムを用いた LED 点灯/消灯制御	学習済
	・ 電流制御抵抗を含めた基本回路動作について確認	学習済
PWM 制御	・ RGB LED およびパッシブブザー制御の実装	実装済
	・ RGB LED のデューティ比制御によるレインボー発光滑らかな色変化を実装	実装済
	・ 周波数制御によるパッシブブザーの音階変化の実装	実装済
	・ 応用として警告音出力について理解を深めた	学習済
Wi-Fi ESP-NOW	・ IEEE 802.11b/g/n(2.4GHz 帯)の基礎理解	学習済
	・ ESP-NOW の接続先湖底による安定通信について確認	実装済
	・ ESP-NOW による Peer to Peer 通信を実装	実装済
	・ 親機/子機間でのデータ送受信及び RSSI 監視を実装	実装済
	・ RSSI を用いた距離推定および通信状態監視を実装	実装済
I2C 通信 LCD 制御	・ I2C 通信の基礎理解	学習済
	・ LCD の制御フローの理解	学習済
	・ I2C 接続 LCD を用いた通信状態及び警告状態の可視化を実装	実装済
	・ ESP32 の I2C 通信によるリアルタイム表示	実装済
	・ ノイズによる I2C 通信異常復帰処理	実装済
インタラクション機能	・ ボタンによる子供向けインタラクション機能の追加	製作途中
しきい値変更	・ ハードウェア側からボタンによるしきい値変更機能の実装	検討中
警告機能の安定化	・ 遮へい物や周辺環境の影響を考慮し、平均化処理やフィルタリング処理	検討中
ノイズ対策	・ ハードウェア側でのノイズ除去や配線設計	検討中

【実装上の課題と対応】

課題	実際に動作確認を行った際、距離が大きく変わっていても RSSI 値が頻繁に変動し、通信状態の判断精度が低下する問題が発生した。
原因	ESP32 の Wi-Fi 仕様や RSSI 特性について、ネット資料やサンプルコードを参考に検証した結果、Wi-Fi 電波は遮へい物による反射・原水の影響を受けやすく、環境要因によって RSSI 値が不安定になることが原因と判明した。
対応	まず誤判定を減らすため、一定時間継続した場合にのみ判定を行う処理や警告音制御を検討した。 その後、サンプルコードや参考資料を基に改善方法を検証し、RSSI 値の一時的な変動による誤判定を防ぐために、一定時間しきい値を超えた場合にのみ発報を行う継続判定処理を導入した。
結果	RSSI 値の急激な変動による誤判定が減少し、通信状態をより安定して判定できるようになった。

課題	Wi-Fi 接続が切断された際、自動で再接続されず、通信状態が復帰しない問題が発生した。
原因	ESP32 の Wi-Fi 接続仕様について調べた結果、一度接続が切断されると、自動で再接続処理を行わない限り、通信が復帰しないことを確認した。そのため RSSI 取得処理も停止し、通信状態の監視を継続できなくなっていた。
対応	まず再接続されない原因を追究し、サンプルコードや参考資料をもとに自動で再接続を行う処理を検討した。 対策として、Wi-Fi の切断イベントを検知した際に自動で再接続を実行するコールバック関数を導入した。
結果	Wi-Fi が切断されても子機から電波が送信されている限り自動的に再接続を行い、RSSI の通信強状態の監視を継続して行えるようになった。

課題	ブザー動作時、LCD がフリーズして復帰しない問題が発生した
原因	動作確認を行った結果、ブザー動作時に高頻度で LCD のフリーズが発生することを確認した。 配線構成や動作タイミングを調査した結果、ブザー動作時に発生するノイズの影響によって LCD 通信が不安定になっている可能性がある判断した。
対応	ノイズ影響を軽減するため、ブザー動作タイミングの見直しや配線の調整をしたが改善が見られなかった。 そのため I2C 通信状態を監視し、一定時間通信エラーが継続した場合は LCD フリーズと判定する処理を実装。 フリーズ検知時には I2C 通信を再初期化し、LCD 初期化コマンドを再送信することで強制復旧を行う構成とした。また、復旧後に直前の表示内容を再描画することで、表示状態を維持できるように改善した。
結果	LCD フリーズ発生時でも自動で通信復旧を行い、表示を継続できるようになった。 これにより、システム全体の安定性と復旧性が向上した。